

438.找到字符串中所有字符的异位词

异位词概念解释

异位词是指由相同字母以不同顺序组成的单词或字符串。

特点

- 包含完全相同的字母
- 每个字母出现的次数完全相同
- 字母的排列顺序可以不同

示例

- "listen" 和 "silent" 是异位词
- "anagram" 和 "nagaram" 是异位词
- "abc" 和 "cab" 是异位词

在 LeetCode 438 中

这道题要求找出字符串s中所有是p的字母异位词的子串的起始索引。

例如：

- 若 $p = \text{"abc"}$ ，则 "acb" 、 "bca" 、 "cab" 等都是 p 的异位词
- 如果在 s 中找到长度为3的子串，包含且仅包含字母a、b、c各一个，不管顺序如何，这个子串就是 p 的一个异位词

判断异位词的常见方法是比较两个字符串中每个字符出现的次数是否完全相同，可以使用计数数组或哈希表实现。

题目

▼ Tags

哈希表 | 字符串 | 滑动窗口

▼ Companies

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

示例 1:

输入: `s = "cbaebabacd"`, `p = "abc"`

输出: `[0,6]`

解释:

起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。

示例 2:

输入: `s = "abab"`, `p = "ab"`

输出: `[0,1,2]`

解释:

起始索引等于 0 的子串是 "ab", 它是 "ab" 的异位词。

起始索引等于 1 的子串是 "ba", 它是 "ab" 的异位词。

起始索引等于 2 的子串是 "ab", 它是 "ab" 的异位词。

提示:

- `1 <= s.length, p.length <= 3 * 10^4`
- `s` 和 `p` 仅包含小写字母

解析

438. 找到字符串中所有字母异位词

这道题要求找出字符串 `s` 中所有 `p` 的异位词的起始索引。可以使用 **滑动窗口** 结合 **字符频率统计** 来解决。

解题思路

1. 使用两个数组记录字符出现的频率：一个记录模式串p的字符频率，一个记录当前窗口的字符频率
2. 维护一个固定长度为p.length()的窗口在s上滑动
3. 每次滑动时，移出一个字符，移入一个字符，更新窗口的字符频率
4. 比较当前窗口和模式串p的字符频率是否相同
5. 如果相同，则当前窗口起始索引就是一个答案

代码实现

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();

        // 边界条件检查
        if (s == null || p == null || s.length() < p.length()) {
            return result;
        }

        // 字符频率计数数组（只考虑小写字母）
        int[] sCount = new int[26];
        int[] pCount = new int[26];

        // 初始化模式串p的字符频率
        for (char c : p.toCharArray()) {
            pCount[c - 'a']++;
        }

        // 初始化第一个窗口的字符频率
        for (int i = 0; i < p.length(); i++) {
            if (i < s.length()) {
                sCount[s.charAt(i) - 'a']++;
            }
        }

        // 比较第一个窗口是否为p的异位词
```

```

    if (Arrays.equals(sCount, pCount)) {
        result.add(0);
    }

    // 滑动窗口
    for (int i = p.length(); i < s.length(); i++) {
        // 移出窗口最左侧的字符
        sCount[s.charAt(i - p.length()) - 'a']--;
        // 移入新的字符
        sCount[s.charAt(i) - 'a']++;

        // 检查当前窗口是否为p的异位词
        if (Arrays.equals(sCount, pCount)) {
            result.add(i - p.length() + 1);
        }
    }

    return result;
}
}

```

示例详解

以输入 $s = \text{"cbaebabacd"}$, $p = \text{"abc"}$ 为例:

1. 初始化:

- $pCount = [1,1,1,0,0,\dots]$ (对应a,b,c的频率)
- 第一个窗口 "cba", $sCount = [1,1,1,0,0,\dots]$
- $Arrays.equals(sCount, pCount)$ 为 true, 添加索引 0 到结果

2. 滑动窗口:

- 移出 'c', 移入 'e': 窗口变为 "bae", $sCount = [1,1,0,0,1,\dots]$
- 比较: 不等于 $pCount$
- 移出 'b', 移入 'b': 窗口变为 "aeb", $sCount = [1,1,0,0,1,\dots]$
- 比较: 不等于 $pCount$
- 依此类推...

- 当窗口变为"bac"（索引6开始）时，sCount = [1,1,1,0,0,...]
- 比较：等于pCount，添加索引6到结果

3. 最终结果：[0,6]

复杂度分析

- 时间复杂度： $O(n)$ ，其中 n 是字符串 s 的长度，我们只需遍历一次字符串
- 空间复杂度： $O(1)$ ，使用了固定大小的数组来存储字符频率